Report

1.0

Customer

Taurus

Smart Account

ABDK Consulting

Contents

1. Document Revisions **3**

2. Introduction **3**

3. System Overview **4**

4. Methodology **4**

5. Findings **6**

5.1. Major 7

5.2. Moderate 7

5.3. Minor 8

6. Conclusion **10**

About Us **11**

Contacts **11**

1. Document Revisions

#	Date	Author	Description

2. Introduction

ALL MODIFICATIONS TO THIS DOCUMENT ARE PROHIBITED. VIOLATORS WILL BE PROSECUTED TO THE FULL EXTENT OF THE U.S. LAW.

ABDK Consulting is pleased to present this technical audit report for Taurus's Smart Account system. As a leading firm in blockchain security and consulting, ABDK brings extensive experience in evaluating the security and robustness of smart contracts and complex blockchain systems.

This audit was conducted by Dmitry Khovratovich and Mikhail Vladimirov, both seasoned experts in the field of cryptography and smart contract security. Their combined expertise ensures a thorough examination of the Smart Account system's codebase and architecture.

Taurus, a prominent entity with a significant presence in its industry, has entrusted ABDK Consulting with this critical security assessment. The Smart Account system is a key component of their technological infrastructure, and ensuring its security is paramount.

ABDK Consulting is committed to providing a comprehensive and unbiased evaluation, identifying potential vulnerabilities and offering actionable recommendations to enhance the overall security posture of the Smart Account system.

IT IS IMPORTANT TO NOTE THAT A SECURITY AUDIT IS NOT A GUARANTEE OF ABSOLUTE SECURITY BUT RATHER A SNAPSHOT OF THE SYSTEM'S SECURITY POSTURE AT A SPECIFIC POINT IN TIME. WHILE WE STRIVE TO UNCOVER ALL POTENTIAL ISSUES, THE EVOLVING NATURE OF BLOCKCHAIN TECHNOLOGY AND DEFI MEANS NEW VULNERABILITIES CAN EMERGE.

3. System Overview

Original files were given in the commit `https://github.com/taurushq-io/smart-wallet-7702/tree/v0.3.0`

Fixes were given in the commit `https://github.com/taurushq-io/smart-wallet-7702/tree/v1.0.0-rc1`

/ (root directory)
TSmartAccount7702.sol

4. Methodology

The methodology utilized by ABDK Consulting is a combination of established audit tactics and methods specifically tuned for the requirements and technological stack of the project to be audited. This approach is flexible and adapted based on the specific project structure, technologies used, and the client's expectations for the security review.

Our audit process incorporates several distinct phases of analysis:

General Code Assessment
The codebase is first reviewed for clarity, consistency, style, and adherence to established best practices. This phase includes checking indentation standards, naming conventions (e.g., following a style guide), appropriate commenting, code duplication, and confusing or irrelevant naming. At this phase, we establish an understanding of the overall system structure and codebase organization.

Entity Usage Analysis
Usages of various entities (contracts, libraries, functions, state variables) defined in the code are analyzed. This includes both internal usages from other parts of the system as well as potential external interactions. We verify that entities are defined with appropriate visibility scopes (internal, external, public, private) and access levels.

At this phase, we understand the overall system architecture and how different parts of the code are related and interact with external interfaces.

Access Control Analysis

18.0ptFor those entities that can be accessed externally or manage sensitive operations (like data submission or role management), access control measures are rigorously analyzed. We check that access control mechanisms are correctly implemented, relevant to the intended function, and robust against circumvention. At this phase, we understand the defined user roles, permissions logic, and what digital assets the system ought to protect.

Code Logic Analysis

The specific code logic of critical functions is analyzed for correctness, efficiency, and security. We verify that the code actually performs its intended function as described in the system overview, that algorithms are optimal for on-chain execution, and that proper data types are used to prevent overflow/underflow issues. We also ensure that external libraries used are up to date and relevant. At this phase, we understand the data structures used and their specific purposes.

We classify all identified issues uncovered during these phases by the following severity levels:

Critical

Directly affects the smart contract functionality and may cause a significant loss of assets (e.g., permanent lock of funds, unauthorized minting, total system compromise) or a complete failure of the platform's core invariants.

Major

Is either a solid performance problem or a sign of significant misuse potential: a slight code modification or environment change may lead to loss of funds or data integrity issues. Sometimes it is an abuse of unclear code behavior which should be double-checked by the client for specific edge-case business logic.

Moderate

Is not an immediate functional problem but rather suboptimal performance in edge cases, an obviously bad code practice, or a situation where the code is correct only in certain expected business flows (lacking robust handling of all possible inputs).

Minor

Covers code style improvements, adherence to best practices that do not immediately present a security risk, and general suggestions for code clarity, gas efficiency, or future-proofing the system.

5. Findings

We have completed our technical audit of the Smart Account codebase and provided the client with a detailed list of recommendations and identified vulnerabilities. The following statistics summarize the issues discovered and their current resolution status at the time of this report's finalization.

	Found	Fixed
Major	1	1
Moderate	3	2
Minor	7	6

Fixed 9 out of 11 issues reported.

5.1. Major

CVF-1: Fixed

Category: **flaw**

As all contracts based on OpenZeppelin's `Initializable` use the same storage slot for initialization status, using such contracts for EIP-7702 delegation could lead to storage collisions.

Recommendation

Use unique storage slots for all purposes, including initialization status.

/TSmartAccount7702.sol

```
    function initialize(address entryPoint_) external initializer {
```

5.2. Moderate

CVF-2: Fixed

Category: **flaw**

This modifier should also require the contract to be already initialized.

/TSmartAccount7702.sol

```
    /// @notice Reverts if the caller is not the EntryPoint.
    modifier onlyEntryPoint() {
        if (msg.sender != entryPoint()) revert Unauthorized();
```

same file

```
    /// @notice Reverts if the caller is neither the EntryPoint nor the account itself.
    modifier onlyEntryPointOrSelf() {
        if (msg.sender != entryPoint() && msg.sender != address(this)) {
```

CVF-3: Fixed

Category: **documentation**

The claim in the comment is incorrect, as the function indeed copied the calldata into memory before forwarding it into the nested call.

Recommendation

Fix the comment, as it is impossible to forward current calldata into another call without copying the data into memory first.

/TSmartAccount7702.sol

```
    ///      Uses assembly to forward calldata directly without copying to memory,
            calldatacopy(m, data.offset, data.length)
```

CVF-4: Acknowledged

Category: **behavior**

This function drops any data returned by the nested call.

Recommendation

Forward the returned data to the caller.

/TSmartAccount7702.sol

```
    function _call(address target, uint256 value, bytes calldata data) internal {
```

CVF-5: Open

Category: **readability**

Forwarding the revert data as is makes it impossible to reliably distinguish error messages from nested calls from error messages from the contract itself.

Recommendation

Wrap the revert data into named error.

/TSmartAccount7702.sol

```
                returndatacopy(m, 0x00, returndatasize())
                revert(m, returndatasize())
```

5.3. Minor

CVF-6: Fixed

Category: **procedural**

Specifying a particular compiler version makes it harder migrating to newer versions.

Recommendation

Specify as `0.8.0`, or as `^0.8.34` if there is something special regarding this particular version.

/TSmartAccount7702.sol

```
pragma solidity 0.8.34;
```

CVF-7: Acknowledged

Category: **procedural**

We didn't review these files.

/TSmartAccount7702.sol

```
import {IAccount} from "account-abstraction/interfaces/IAccount.sol";
import {PackedUserOperation} from
"account-abstraction/interfaces/PackedUserOperation.sol";
```

CVF-8: Fixed

Category: **efficiency**

These errors could be made more useful by adding certain parameters into them

/TSmartAccount7702.sol

```
    error Unauthorized();
    error EmptyBytecode();
```

CVF-9: Fixed

Category: **efficiency**

Solidity compiler is smart enough to precompute constant hash expressions.

Recommendation

Use a hash expression instead of a hardcoded hash value.

/TSmartAccount7702.sol

```
    /// @dev ERC-7201 namespaced storage slot for `EntryPointStorage`.
    ///
keccak256(abi.encode(uint256(keccak256("smartaccount7702.entrypoint")) -
1)) & ~bytes32(uint256(0xff))
    bytes32 private constant ENTRY_POINT_STORAGE_LOCATION =
        0x38a124a88e3a590426742b6544792c2b2bc21792f86c1fa1375b57726d827a00;
```

CVF-10: Fixed

Category: **documentation**

It is a good practice to put a comment into an empty block to explain why the block is empty.

/TSmartAccount7702.sol

```
    receive() external payable {}
```

same file

```
    fallback() external payable {}
```

CVF-11: Fixed

Category: **typing**

The version number should be a named constant.

/TSmartAccount7702.sol

```
        return "0.3.0";
```

CVF-12: Fixed

Category: **typing**

Interface signatures should be named constants.

/TSmartAccount7702.sol

```
            || interfaceId == bytes4(0x1626ba7e) // ERC-1271
```

```
            || interfaceId == bytes4(0x77390001) // ERC-7739
            || interfaceId == bytes4(0x150b7a02) // IERC721Receiver
            || interfaceId == bytes4(0x4e2312e0) // IERC1155Receiver
            || interfaceId == bytes4(0x01ffc9a7); // ERC-165
```

6. Conclusion

The audit of the Smart Account contract identified a major issue, CVF-1, related to potential storage collisions when using EIP-7702 delegation. This issue arises because contracts based on OpenZeppelin's Initializable utilize a shared storage slot for their initialization status. The recommendation is to implement unique storage slots for all contract functionalities, including the initialization status, to prevent such collisions.

Several other medium-severity issues were also noted. These include a modifier that should enforce initialization before execution, a discrepancy in documentation regarding function behavior with calldata, and a function that discards return data from nested calls. Additionally, concerns were raised about the forwarding of revert data, which can obscure error origins, and a general recommendation to wrap revert data into named errors for clearer error handling.

All major issues were fixed.

ABDK
Consulting

About Us

Established in 2016, is a leading service provider in the space of blockchain development and audit. It has contributed to numerous blockchain projects, and coauthored some widely known blockchain primitives like Poseidon hash function.

The ABDK Audit Team, led by Mikhail Vladimirov and Dmitry Khovratovich, has conducted over 300 audits of blockchain projects in Solidity, Rust, Circom, C++, Javascript, and other languages.

Contacts

EMail

info@abdkconsulting.com

Website

abdk.consulting

X (former Twitter)

@ABDKConsulting

LinkedIn

company/abdk-consulting